Comparing Sequence Modeling Architectures for EMG-Based Text Decoding

Zuyan Tao
zuyantao@g.ucla.edu
306603229

Mingyu Zhu
tonyzhu@g.ucla.edu
705646591

Danqi He
hedanqi77@g.ucla.edu
606628596

Guanhua Chen
gchen2002@g.ucla.edu
706779556

Abstract

This paper studies the task of decoding text input from forearm electromyography (EMG) signals using the *emg2qwerty* dataset. The goal is to investigate how different sequence modeling architectures influence decoding performance while keeping the data preprocessing pipeline, feature extraction module, training strategy, and decoding method consistent. We compare three encoder architectures: the TDSConv encoder used in the official baseline, as well as two alternative designs, CNN + Transformer and CNN + BiLSTM.

Experimental results show that both CNN + Transformer and CNN + BiLSTM significantly outperform the convolution-only TDSConv baseline when evaluated using CTC greedy decoding. Among the three architectures, CNN + BiLSTM achieves the lowest Character Error Rate (CER) on both validation and test sets. Based on this result, we further explore several factors affecting performance using the CNN + BiLSTM model, including beam search decoding, the removal of data augmentation, and the reduction of electrode channels. Beam search decoding further improves CER, while reducing the number of electrode channels leads to a noticeable performance drop.

These findings suggest that combining convolutional feature extraction with explicit temporal modeling provides a more effective architecture for EMG-based text decoding tasks.

1 Introduction

The *emg2qwerty* dataset provides a standardized research task for using forearm EMG signals to predict keyboard input [6]. In this task, the model takes multi-channel EMG spectrogram features as input and predicts the corresponding character sequence, and model performance is usually evaluated using the Character Error Rate (CER). The official codebase provides a baseline model based on the TDSConv encoder trained with the CTC loss function [6].

In this project, our goal is to reduce CER by exploring different sequence modeling structures under the *emg2qwerty* framework. Specifically, we focus on the impact of different encoder architectures on model performance while maintaining consistent data processing procedures, feature extraction modules, training strategies, and decoding methods.

To conduct a controlled comparison, we keep the data preprocessing pipeline, feature extraction module, training configuration, and decoding method consistent with the official implementation. The only component that changes across experiments is the sequence encoder architecture.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

Under this experimental setting, we compare three different sequence modeling methods: the TDSConv encoder in the official implementation, and two alternative architectures — CNN + Transformer and CNN + BiLSTM. The former uses a self-attention mechanism to model relationships between different time steps in the sequence, while the latter uses a bidirectional recurrent network to capture contextual information in the sequence.

All experiments are performed on data from the same subject (ID #89335547) in the *emg2qwerty* dataset. This setting reduces additional variability caused by cross-subject differences, allowing us to more clearly analyze the impact of different model architectures on decoding performance.

Through these experiments, we aim to systematically analyze the performance differences of different sequence modeling architectures in EMG input decoding tasks, and provide insights for model selection in future EMG-based input system design.

2 Methods

To ensure fairness in the experiments, we only replaced the encoder architecture of the model under the same data processing pipeline, training strategies and decoding methods.

The overall system consists of an EMG feature extraction module, a sequence encoder and a CTC decoding module. The three models are consistent in input preprocessing, feature mapping, and output layer design, with only differences in sequence modeling modules.

Figure 1 shows the overall data flow structure shared by the three models, while Figure 2 illustrates the architectures of the three encoder structures.

2.1 Data Processing Pipeline

The three models in this study share the same data processing and training pipeline, except for differences in the structure of the sequence encoder. The overall data flow structure is shown in Figure 1.

The input to the system is a multi-channel spectral representation (EMG spectrogram) calculated from surface electromyography signals (EMG). Each input sample can be represented as a time-varying sequence of spectral features. In order to reduce the scale differences between different channels and stabilize the input distribution, the spectral features are first normalized using spectrogram normalization to keep the statistical distributions across channels consistent, thereby improving the stability of model training.

The normalized features are then input to the Multi-Band Rotation-Invariant MLP module. This module performs nonlinear mapping of spectral features in different frequency bands and learns feature representations with rotation invariance. Specifically, the input spectrum is first divided into multiple frequency bands, and each frequency band is projected through a multilayer perceptron that shares parameters, thereby obtaining a more stable representation while maintaining band structure information.

After multi-band feature mapping, the features of each band will be flattened in the channel dimension to form a unified time series representation. After this operation, the input features can be represented as a three-dimensional tensor: (T, B, 768) where T represents the number of time steps, B represents the batch size, and 768 represents the expanded feature dimension.

The expanded sequence features are then input into a sequence encoder for timing modeling. In this study, we compared three different sequence modeling structures: TDSConv, CNN + Transformer, and CNN + BiLSTM. Each encoder receives an input sequence of the same dimension and outputs a new time series representation while keeping the time dimension unchanged.

The hidden representation output by the encoder is then mapped to the character vocabulary space through a linear projection to obtain the character probability distribution corresponding to each time step. The output dimension is (T, B, |V|), where |V| represents the size of the character vocabulary.

During the training phase, the model is optimized using Connectionist Temporal Classification (CTC) loss [3]. CTC allows models to learn the mapping from continuous EMG signals to character sequences without explicit frame-level alignment annotations. During the inference phase, the logits

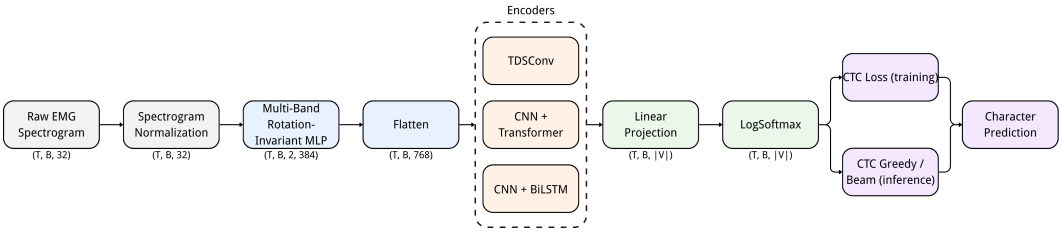

Figure 1: Overview of the EMG-to-text pipeline.

output by the model are decoded through CTC decoding to generate the final character prediction sequence.

2.2 Baseline Model: TDSConv Encoder

The TDSConv model was used as the baseline for this study. This architecture uses Time-Depth Separable Convolution for sequence modeling, capturing local temporal dependencies by performing convolution operations in the time dimension.

Specifically, the TDSConv encoder consists of multiple TDS blocks. Each block contains the following components: temporal depthwise convolution, pointwise convolution, residual connection and layer normalization.

Depthwise convolution is responsible for extracting local temporal features in the temporal dimension, while pointwise convolution is used for cross-channel feature fusion. Residual connection and normalization operations are used to stabilize the training process.

Multiple TDS blocks are stacked to form a complete sequence encoder, and its inputs and outputs maintain the same time dimension: mapping the input representation from (T, B, 768) to (T, B, H), where H denotes the encoder hidden dimension. The schematic structure is shown at the top of Figure 2.

2.3 CNN + Transformer Encoder

The second model uses a combined structure of CNN + Transformer for sequence modeling. The structure first extracts local EMG feature patterns through a convolutional network, and then uses a Transformer encoder to model long-range dependencies in the sequence. The architecture consists of two stages: Temporal CNN Feature Extraction and Transformer Sequence Modeling.

In the temporal CNN feature extraction stage, a one-dimensional convolutional layer is used to extract local features from the time series. Each convolution module contains: 1D convolution layer, batch normalization, ReLU activation, and dropout. This stage is mainly used to extract local temporal patterns and reduce the impact of noise on the model.

In the transformer sequence modeling stage, the CNN output is then input to the Transformer encoder. Each transformer encoder layer contains multi-head self-attention, a position-wise feed-forward network, residual connections, and layer normalization [7]. In addition, a positional encoding is added before the Transformer input to provide positional information.

The schematic structure is shown in the middle of Figure 2.

2.4 CNN + BiLSTM Encoder

The third model uses a combined structure of CNN + BiLSTM for sequence modeling. The architecture first uses a convolutional network to extract local EMG features, and then uses a bidirectional recurrent neural network to model temporal context information.

Similar to the previous model, the input sequence is first subjected to local feature extraction through a one-dimensional convolution module. The convolution features are then input into a Bidirectional LSTM (BiLSTM) network. BiLSTM processes sequences in both forward and reverse temporal

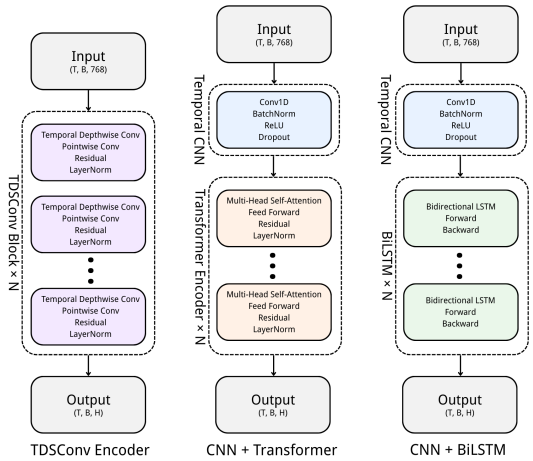

Figure 2: Comparison of the three encoder architectures used in this study

directions, resulting in bidirectional contextual representations [5]. Multiple BiLSTM layers are stacked to form a complete sequential encoder while preserving the temporal dimension.

The schematic structure is shown in the bottom of Figure 2.

2.5 Universal Training Hyperparameters

To ensure fair comparison between different model structures, this study maintains consistent data preprocessing procedures, training strategies, and decoding configurations in all experiments, and only changes the structure of the sequence encoder.

In terms of input feature processing, all models adopt the same feature extraction method. The EMG spectrogram is first subjected to spectrogram normalization and then input to the multi-band rotation-invariant MLP module for feature mapping. The feature extraction module uses an input feature dimension of 528, and the spectrum is divided into two frequency bands. Each frequency band is projected through an MLP with a hidden dimension of 384. After band-wise feature mapping and flattening along the channel dimension, a time series representation with dimension (T, B, 768) is obtained, which serves as the unified input for all encoder architectures.

In terms of model training, all experiments use AdamW as the optimizer and adopt a learning rate schedule combining linear warmup and cosine annealing. At the beginning of training, a warmup with 10 epochs was used to gradually increase the learning rate to improve training stability, and then the learning rate gradually declined according to the cosine schedule. The maximum number of training rounds for all models is set to 150 epochs, and mixed-precision training is used to reduce memory consumption and improve training efficiency.

In terms of output layer and decoding strategy, the three models also remain completely consistent. The encoder output is mapped to the character vocabulary space through the same linear projection layer and trained using CTC loss. Greedy decoding is used for alignment learning during the training phase, and beam search decoding is used during inference to obtain more stable character prediction results. All models share the same character vocabulary and use the same CTC blank token.

In addition, all experiments used completely consistent data partitioning and data processing methods. EMG signals are segmented using the same window segmentation strategy, and the same data segmentation is used during the training, validation and testing phases. Sequence padding, batch building, and data loading processes are also consistent across all models.

By fixing data processing, training configuration, and decoding strategy, this study ensures that the only variable in the experiment is encoder architecture. This setup allows the comparison to directly reflect differences in the sequence modeling structure itself.

3 Results

Table 1: Validation and test performance of different encoder architectures and ablation settings.

Model / Configuration	Split	CER (%)	DER (%)	IER (%)	SER (%)
TDSConv Baseline (Greedy)	Validation	19.10	1.77	4.87	12.45
	Test	22.39	2.23	4.63	15.54
CNN + Transformer (Greedy)	Validation	14.71	2.28	3.12	29.30
	Test	16.57	2.49	3.42	10.65
CNN + BiLSTM (Greedy)	Validation	14.44	1.33	3.23	9.88
	Test	15.06	1.15	2.29	11.63
CNN + BiLSTM (Beam Search)	Validation	9.70	2.37	2.46	4.87
	Test	11.80	2.58	2.92	6.30
CNN + BiLSTM (w/o augmentation)	Validation	14.49	1.17	3.23	10.08
	Test	15.34	1.17	2.81	11.37
CNN + BiLSTM (8 electrode channels)	Validation	18.37	1.79	4.14	12.43
	Test	21.37	2.17	4.35	14.85

We use Character Error Rate (CER) as the primary evaluation metric, where a lower value indicates better decoding performance. The validation and test results of all configurations are summarized in Table 1.

3.1 Encoder Architecture Comparison

We first compare the three encoder architectures trained with the same configuration and evaluated using greedy CTC decoding. The results are reported in Table 1.

For the **TDSConv baseline**, the model achieves a CER of 19.10% on the validation set and 22.39% on the test set.

For the **CNN + Transformer** architecture, the CER decreases to 14.71% on the validation set and 16.57% on the test set.

For the **CNN + BiLSTM** architecture, the CER further decreases to 14.44% on the validation set and 15.06% on the test set.

Among the three architectures trained with greedy decoding, CNN + BiLSTM obtains the lowest CER. Therefore, in order to reduce computational cost in subsequent experiments, we select the CNN + BiLSTM architecture as the base model for the following studies.

3.2 Beam Search Decoding

After selecting CNN + BiLSTM as the base architecture, we further evaluate the effect of beam search decoding during inference.

Using greedy decoding, the CNN + BiLSTM model obtains a CER of 14.44% on the validation set and 15.06% on the test set. When beam search decoding is applied, the CER decreases to 9.70% on the validation set and 11.80% on the test set.

3.3 Effect of Data Augmentation

We also investigate the impact of data augmentation on the CNN + BiLSTM architecture. In the default configuration, several augmentation methods are applied during training.

With the full augmentation pipeline enabled, the CNN + BiLSTM model achieves a CER of 14.44% on the validation set and 15.06% on the test set. When these augmentation techniques are removed while keeping all other settings unchanged, the model obtains a CER of 14.49% on the validation set and 15.34% on the test set.

For reference, we also conducted the same experiment using the CNN + Transformer architecture. The test CER changes from 16.57% to 16.98% when the augmentation modules are removed.

3.4 Effect of Electrode Channel Reduction

Finally, we examine how the number of electrode channels affects decoding performance. In this experiment, the number of electrode channels is reduced from 16 to 8 while keeping the CNN + BiLSTM architecture and other training settings unchanged.

Under this configuration, the CER increases to 18.37% on the validation set and 21.37% on the test set, compared with 14.44% and 15.06% obtained using all 16 channels.

All results are summarized in Table 1.

4 Discussion

Advantages over TDSConv Compared with the baseline TDSConv encoder, the two architectures proposed in this paper provide stronger expressive capabilities for temporal modeling. TDSConv mainly relies on convolution operations to extract local features along the temporal dimension, while both models in this study introduce additional sequence modeling modules after convolutional feature extraction, allowing the temporal relationships in EMG signals to be modeled more effectively.

In both architectures, the convolutional layer first serves as a front-end feature extraction module to encode local activation patterns in the EMG signal. EMG signals typically manifest as short-term changes in muscle activation, and such local patterns often correspond to specific finger movements or keystroke actions. Extracting these local features through convolution provides more stable and informative representations for subsequent sequence modeling modules.

After the convolutional feature extraction stage, the two models adopt different mechanisms for temporal modeling. The CNN + Transformer architecture models interactions between arbitrary time steps through a self-attention mechanism, while CNN + BiLSTM propagates contextual information along the temporal dimension using recurrent connections. Both approaches enable the model to capture longer-range temporal dependencies based on convolutional features, providing greater flexibility in sequence modeling than the convolution-only TDSConv encoder.

Limitations of Transformer in Long Sequence Inference Although the CNN + Transformer architecture provides strong expressive capability, we observe that its performance is sensitive to differences in sequence length between the training and inference phases. During training, EMG signals are typically segmented into fixed-length windows, and the positional representations learned by the Transformer are therefore concentrated within this length range. However, during inference, the model often needs to process longer sequences of continuous EMG signals.

This mismatch between training and inference sequence lengths introduces issues related to positional encoding. Standard absolute positional encodings are usually learned only within the positional range observed during training. When the inference sequence length significantly exceeds the training length, the model needs to handle positional indices that were not encountered during training, which may lead to unstable attention distributions and affect sequence modeling and character alignment.

A common solution is to apply window-based decoding or sliding-window attention during inference so that the sequence length remains consistent with the training distribution [1]. However, this approach requires additional stitching across different windows and may limit the model's ability to capture long-range dependencies across window boundaries.

Why CNN+BiLSTM Performs Better In contrast, the CNN + BiLSTM architecture shows stronger suitability for EMG typing tasks. Surface EMG signals are inherently continuous time-series signals, and their dynamics exhibit strong temporal dependencies [2]. Recurrent neural networks, particularly bidirectional LSTMs, explicitly propagate contextual information along the temporal dimension and are therefore well suited for modeling such sequential signals.

In the CNN + BiLSTM architecture, convolutional layers first extract local temporal patterns from the EMG signal, while BiLSTM models longer-range contextual dependencies based on these features. Since BiLSTM processes sequences in both forward and backward temporal directions, its hidden

representations can incorporate both past and future context information, which is particularly beneficial for the sequence alignment process in CTC-based training.

This combination of local feature extraction and bidirectional temporal modeling is well suited for tasks that map continuous signals to symbolic sequences. Similar CNN + RNN architectures have also been widely adopted in related domains such as speech recognition and handwriting recognition [4], where both local patterns and long-range temporal dependencies need to be modeled.

Why data augmentation does not improve CER significantly One possible reason why data augmentation does not improve CER significantly is that we are training and testing on data from a single user. For instance, the RandomBandRotation module is intended to simulate different band positions across different participants. Since we only use one person's data in this project, the band position is relatively fixed. In this setting, adding random rotations may act more like noise and interfere with the model's ability to capture this specific participant's sEMG patterns. If the same experiment were conducted in a multi-user setting, this module might contribute more to overall performance.

Moreover, the TemporalAlignmentJitter module is designed to simulate differences in reaction time. However, our loss function, CTC, is already tolerant to unsegmented temporal sequences and timing variability. As a result, the benefit brought by this augmentation may be limited in our current setting.

Importance of electrode channels As demonstrated previously, the model's performance degraded significantly when the number of electrode channels was reduced from 16 to 8, with the test CER increasing from 15.06 to 21.37. This result suggests that the full set of sensors is important for capturing the spatial information needed for accurate typing prediction. From a biomechanical perspective, keyboard typing involves coordinated muscle activity across the hand and forearm. To preserve this complex neuromuscular information, the full set of electrode channels on the band appears necessary for capturing the subtle sEMG patterns associated with typing.

5 Summary

Focusing on the task of EMG input-to-text decoding, this paper systematically compares the performance of different sequence modeling structures on the *emg2qwerty* dataset. While maintaining consistency in the data preprocessing pipeline, feature extraction module, training strategy, and decoding method, we focus on the impact of different encoder architectures on model performance. Specifically, we compare the TDSConv encoder used in the official implementation with two alternative architectures: CNN + Transformer and CNN + BiLSTM.

Experimental results show that when using CTC greedy decoding, both architectures that introduce stronger sequence modeling capability perform significantly better than the TDSConv baseline model that relies only on convolution structures. Among them, CNN + BiLSTM achieves the lowest CER on both the validation and test sets, and is therefore selected as the base architecture for subsequent experiments. On this basis, we further analyze the impact of several factors on model performance. First, using beam search decoding during the inference stage further reduces CER, indicating that a more comprehensive sequence search under the CTC framework can lead to more stable character prediction results. Second, by removing the data augmentation module for comparison, we observe that data augmentation provides only limited improvement under the current single-user setting. Finally, a significant decline in model performance is observed when the number of electrode channels is reduced, indicating that multi-channel EMG signals play an important role in capturing spatial information related to hand and forearm muscle activity.

Overall, this study shows that introducing explicit sequence modeling capability can effectively improve performance in EMG input decoding tasks. Among the evaluated architectures, CNN + BiLSTM shows the most stable performance under the current experimental setting and provides a useful reference for future EMG-based input system design.

References

[1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020.

[2] Dario Farina, Roberto Merletti, and Roger M. Enoka. The extraction of neural strategies from the surface emg: 2004–2024. *Journal of Applied Physiology*, 138(1):121–135, 2025. PMID: 39576281.

[3] Alex Graves, Santiago Fernandez, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 369–376, 2006.

[4] Awni Hannun, Carl Case, Jared Casper, Bryan Catanzaro, Greg Diamos, Erich Elsen, Ryan Prenger, Sanjeev Satheesh, Shubho Sengupta, Adam Coates, and Andrew Y. Ng. Deep speech: Scaling up end-to-end speech recognition, 2014.

[5] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 11 1997.

[6] Viswanath Sivakumar, Jeffrey Seely, Alan Du, Sean R Bittner, Adam Berenzweig, Anuoluwapo Bolarinwa, Alexandre Gramfort, and Michael I Mandel. emg2qwerty: A large dataset with baselines for touch typing using surface electromyography, 2025.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.